



SAPIENZA
UNIVERSITÀ DI ROMA

DEPARTMENT OF COMPUTER, CONTROL AND MANAGEMENT
ENGINEERING

Neural Model Predictive Control on a Unicycle

INTELLIGENT CONTROL

Professors:

Alessandro Giuseppe

Students:

Flavio Maiorana

Flavio Volpi

Academic Year 2023/2024

Contents

Todo list	2
1 Introduction	3
2 Model	3
2.1 Unicycle kinematic model	3
2.2 Unicycle Neural model	4
2.2.1 Architecture	4
2.2.2 Dataset	4
2.2.3 Training and loss	5
2.2.4 Extra: PINN	5
3 Model Predictive Control	6
3.1 Cost function	6
3.2 Constraints	7
4 Implementation	7
4.1 Simulator	8
4.2 Model	8
4.3 MPC	9
5 Results	10
5.1 Derivative Prediction	10
5.2 State Prediction	12
5.3 State Prediction with Physics Loss	14
6 Conclusions	15

Todo list

Figure: text	12
Figure: text	12
Figure: Loss plot for the state prediction task	12
Figure: Unsuccessful evaluation on a test sample	12
Figure: text	14
Figure: text	14
Figure: Loss plot for the state prediction task	14
Figure: text	15
Figure: text	15
Add a paragraph on the future work	16

1 Introduction

Model Predictive Control (MPC) has emerged as a powerful control strategy in robotics due to its ability to handle constraints and optimize system behavior over a prediction horizon. When combined with neural networks, which can learn complex system dynamics, it creates a promising framework for controlling nonlinear systems.

This report presents the implementation and analysis of a Neural Model Predictive Control (NMPC) scheme applied to a unicycle robot model. The unicycle serves as an excellent first-test case due to its nonholonomic constraints and nonlinear dynamics, making it both challenging and representative of real-world robotic systems, but at the same time its low complexity makes it ideal for testing without too many complications.

Our approach integrates deep learning techniques to approximate the unicycle's dynamics, which are then utilized within an MPC framework to generate optimal control inputs. We explore the effectiveness of this method in trajectory tracking tasks, comparing it with traditional control approaches.

The following sections detail the theoretical foundations, implementation specifics, and experimental results of our neural MPC system. We also discuss the challenges encountered and potential improvements for future work.

2 Model

The model must be well-defined to be used in NMPC, in such a way to obtain reliable predictions of system behavior in response to various control actions. The usage of a data-driven approach, such as neural networks, in the vehicle modeling phase has the advantage of not modeling all the nonlinear physical effects, which can be also very complex. In fact, neural networks have the capability to accurately model them. Neural networks are universal approximators, and they can be used both for designing the entire dynamic model of a system and approximating the disturbances and noises a system can be affected. Our purpose is to approximate the entire kinematic model of an unicycle with a neural model, due to its low complexity. In this section we will start describing briefly the kinematic model of a unicycle, and then we will describe the neural architecture used to approximate it.

2.1 Unicycle kinematic model

The kinematic model of the unicycle is a simplified representation of the movement of the vehicle in terms of position and orientation. It assumes there will be no lateral deformations. The state space is represented by its position and orientational

$$[x, y, \theta] \tag{1}$$

while the input actions are represented by longitudinal and angular velocities the vehicle can assume

$$[v, \omega] \quad (2)$$

So, the motion of the unicycle can be described through the following equations:

$$\dot{x}(t) = v(t) \cos(\theta(t)) \quad (3)$$

$$\dot{y}(t) = v(t) \sin(\theta(t)) \quad (4)$$

$$\dot{\theta}(t) = \omega(t) \quad (5)$$

$$(6)$$

2.2 Unicycle Neural model

To model the kinematic model of the unicycle, a simple architecture is needed, but not too simple, in order to captivate the dynamics of the system.

2.2.1 Architecture

The neural network used to model the kinematic model of the unicycle is a simple MLP architecture, with two hidden layers of 128 neurons. The network has two inputs which are concatenated together, the current state of the unicycle (dimension 3) and the current applied actions on the vehicle (dimension 2), so it must be of dimension 5 and is represented by

$$X = [x, y, \theta, v, \omega] \quad (7)$$

As for the output, we tried two different strategies: the first one is to have the output of the network to be the next state of the unicycle, so it is a vector of dimension 3, which represents the position and orientation of the vehicle at the next time-step. The second strategy is to have the output of the network to be the dynamics of the system, so it is a vector of dimension 3, which represents the velocity of the vehicle in the x-y direction and the angular velocity.

In the first case, the output at time k is

$$Y_k = [x_{k+1}, y_{k+1}, \theta_{k+1}] \quad (8)$$

In the second case, the output at time k is

$$Y_k = [\dot{x}, \dot{y}_k, \dot{\theta}_k] \quad (9)$$

2.2.2 Dataset

The dataset was synthetically generated by using the nominal model of the robot and generating N trajectories, each composed by K steps, where the inputs are maintained constant for a number of M steps. This choice was taken because it was noticed that the interval between consecutive time-steps is too small,

therefore the changes about the state of the unicycle are small too, and the network was not able to approximate correctly the motion of the vehicle. On the contrary, having the same input for multiple steps allows to understand in a better way the dynamics of the system subject to specific inputs.

Each sample is not just a pair state-action at step t , but it is a window of dimension k which contains all pairs state-action from time-step t till $t + k$ (less the last action a_{t+k} which is useless), so it is in the form

$$(s_t, a_t, s_{t+1}, a_{t+1}, \dots, s_{t+k-1}, a_{t+k-1}, s_{t+k}) \quad (10)$$

This allows to have a larger window of evolution of the system with respect to just the motion executed in one single step. What really matters, of course, are the labels of our dataset. They are either the derivatives, or the successive states.

An exemplary configuration of the dataset is the following:

- $N = 10.000$
- $K = 20$
- $M = 5$

Each sub-trajectory ends with the first state of the successive sub-trajectory. This way we are able to learn the dynamics of the system under different time conditions, in order to observe better the changes and try the network to learn better.

We divided the dataset in training, validation and test set, such that the validation set can guide the training process, while the test set can validate the results, in thi case also graphically.

2.2.3 Training and loss

As said before, we tried two different strategies to train the network. In the first case, the network has to learn the derivative of the system, given a state and an input. Thus, we simply compare the output of the network with the nominal model, imposing a mean squared error loss:

$$loss = \frac{1}{k} \sum_{i=t}^{t+k} (\bar{s}_i - s_i)^2 \quad (11)$$

In the second case, the network has to learn the next state of the system, thus we still use a mean squared error loss, but this time on the states.

2.2.4 Extra: PINN

We also tried to make our network a Physics-Informed Neural network, which incorporates some prior physical knowledge through the addition of a specific term in the loss function. In detail, the neural architecture is the same explained

in subsubsection 2.2.1; what changes is that, if we have already some information about the dynamics of the system in the form of differential equations, we can add a physics loss term telling how much the approximated dynamics reflects the known one. In this way, the loss function becomes:

$$loss = \frac{1}{k} \sum_{i=t}^{t+k} (\bar{s}_i - s_i)^2 + \lambda \frac{1}{M} \sum_j^M (g(x_j) - f(x_j))^2 \quad (12)$$

where g is the known differential equation, while f is the neural approximator. The M points on which the physics loss is computed are called **collocation points**. These must not be samples in the training datasets, but they can be any points we want, of which we can compute the dynamics. This can be a useful approach if we don't have a large amount of training data and if we already know something about the problem which is not related directly to data.

3 Model Predictive Control

Model Predictive Control (MPC) is an optimal control technique where the actions are chosen solving an optimization problem, composed by a set of constraints and an objective function to be minimized over a finite horizon.

3.1 Cost function

Our objective is to minimize at each step the error with respect to the reference trajectory, both in traslational and orientational dimension, while also maintaining the module of the input actions as low as possible. A terminal state term is present too, where the traslational and orientational errors of the last horizon step are tried to be minimized. Thus, the cost function is composed by the following three terms:

1. **Spatial error:** at each step of the horizon, errors along x-axis, y-axis and error of orientation are summed together

$$J_{sp} = W_{e_x} e_x + W_{e_y} e_y + W_{e_\theta} e_\theta \quad (13)$$

where

$$e_x = x - x_{ref} \quad (14a)$$

$$e_y = y - y_{ref} \quad (14b)$$

$$e_\theta = \theta - \theta_{ref} \quad (14c)$$

2. **Terminal state:** to make sure the final stage is a good one, we add a penalty on errors along x-axis, y-axis and error of orientation in the terminal state

$$J_{term} = W_{e_{x_{term}}} e_{x_{term}} + W_{e_{y_{term}}} e_{y_{term}} + W_{e_{\theta_{term}}} e_{\theta_{term}} \quad (15)$$

3. **Input:** it penalizes high values of input controls

$$J_u = W_v v + W_\omega \omega \quad (16)$$

3.2 Constraints

The constraints of the NLP concerns some limitations on the state of the vehicle model and on the dynamics of the system. In detail, we have just two constraints:

1. **Initial state:** the initial state of the unicycle model must be the current measured state. Therefore, defining the vehicle state as x :

$$x_0 = x_{measured} \quad (17)$$

2. **Dynamics:** given the unicycle state at a certain time-step as x_t , an input at the same time-step as u_t and the dynamic model of the system defined as f_{expl} , we establish that the next state is obtained by applying the input at the given state according to dynamic model:

$$x_{t+1} = f_{expl}(x_t, u_t) \quad (18)$$

4 Implementation

The NMPC is developed using Acados, a software package that provides solvers for nonlinear optimal control. It is based on the symbolic framework CasADi, and provides a Python interface at an higher level. CasADI is a software framework for automatic differentiation, nonlinear optimization and optimal control. It is implemented in C++ and has also a Python interface.

The neural model has been developed using PyTorch, an open source deep learning library based on Python.

The linking between the MPC and the neural model has been possible using the L4Casadi framework, which allows the integration of Pytorch-learned models within CasADI and Acados, and provides also an hardware acceleration for learned components.

Acados was used for three main purposes:

- Modeling
- Simulation through numerical integration
- Formulation and solving of optimization problems within MPC

The software of this project is based mainly on three components:

Simulator Matplotlib plot to show everything graphically in real-time.

Model Expressed formally according to Acados syntax, the ODE is expressed in implicit form.

Controller Expressed as an Acados linear program, with a linear cost function and linear constraints; it returns a single control action based on the current state and the predicted states for the whole horizon.

4.1 Simulator

The main goal of the simulator is to visualize the reference trajectory that evolves in time, and to simulate the unicycle movement through the control inputs and integration of the model. The complete parametrized implementation of a simulation can be found at `trajectory_tracking.py`, which takes in input the interested robot (in our case the unicycle, but it can be extended to other robots), the used controller and the reference trajectory, and it generates the entire simulation loop. The simulation loop is summarized in pseudocode 1, where the whole cycle is encapsulated in the function

```
run(self, N, animate, save)
```

which takes the total number of steps N , a boolean to express if the simulation must be visualized and animated through a proper function `animate`, and another boolean if the animation video must be saved. The animation is guaranteed by `FuncAnimation` of Matplotlib.

The trajectories the robot can track are defined inside `trajectory.py`. A trajectory must contain a function `update(self, t)` which declares at time-step t its position, velocity, acceleration, orientation and angular velocity.

Algorithm 1 Simulation Loop

```

1:  $dt \leftarrow 0.05$  s ▷ Time step
2: while number of steps is not exceeded do
3:   measure car state  $x$ 
4:   action  $a \leftarrow command(x)$  ▷ From MPC
5:   move forward for  $dt$  under action  $a$ , updating state  $x$ 
6:   update plot and save state and action
7: end while

```

4.2 Model

An important aspect is how the models are actually discretized. What happens is that, conceptually, the model is still expressed as a symbolic function through CasADI, as if it was a continuous function. Acados helps in doing this, providing an high level interface through the class `AcadosModel`. Instantiating an object of this class, we have to deliver it state, inputs and derivative of the state as CasADI symbolic variables, and also implicit or explicit ODE in the form of CasADI expression, according to the integrator type wants to be used. All these information are necessary to the class to automatically generate the CasADI

function. The formulation of the real unicycle model according to Acados policy can be found in `unicycle_kin_acados.py`.

In the same way, the neural model of the robot is expressed through the `AcadosModel`. The only distinction lies in the expression of the explicit (or implicit) ODE, which recalls the neural approximator thanks to **L4Casadi** [?]. In fact, it is possible to create an instance of the class `L4CasADi`, which takes in input the Pytorch tensor; at this point the forward pass of the neural network can be executed, calling the `L4CasADi` instance with the symbolic variables as inputs. In this way, a symbolic CasADi expression is returned, representing the explicit dynamics of the system. The detailed formulation of the neural unicycle model can be found in `unicycle_kin_acados_neural.py`

4.3 MPC

The implementation of the MPC for both real and neural unicycle models is the same, because it is enclosed inside a wrapper class called `ModelPredictiveController`. This class contains two methods:

- `_init_ocrp()`: initialize the optimization problem and its solver.
In Acados, the formulation of the optimization control problem is done thorough the class `AcadosOcp`, where the full problem can be described, specifying the dimension of the problem, the interested `AcadosModel` (see subsection 4.2), the cost function in matricial form, the constraints, the type of solver to use, integrator type the solver must use, and several other options.
Once the formulation is complete, an acados solver can be effectively created using the `AcadosOcpSolver` class. This is a communication bridge with the acados OCP solver in C
- `command(robot: Unicycle, reference: Trajectory)`: instantiate the optimization problem in numerical form and solves it. It takes in input the robot model and the reference trajectory, and returns the action to be applied, the next state, the reference position and the error between the prediction and the baseline. What happens inside this function in details: the solver allows to set numerical data inside the constraints and the cost function. Then, the built-in function in the solver `solve_for_x0(state)` is called, returning the best action can be applied in *state*.

Cost Weights	Value
e_x	3
e_y	3
e_ψ	2
v	1
w	1
$e_{x,\text{term}}$	5
$e_{y,\text{term}}$	5
$e_{\psi,\text{term}}$	3

Table 1: Cost Weights

Constraints	Value
$x_{\min}$	-2
$x_{\max}$	2
$y_{\min}$	-2
$y_{\max}$	2
$\psi_{\min}$	-2
$\psi_{\max}$	2
$v_{\min}$	-3
$v_{\max}$	3
$w_{\min}$	-3
$w_{\max}$	3

Table 2: Constraints

5 Results

In this section we will report some results obtained by training the neural model and applying it to an MPC based trajectory tracking problem.

5.1 Derivative Prediction

In figure 1 we can observe that the loss converges pretty quickly.

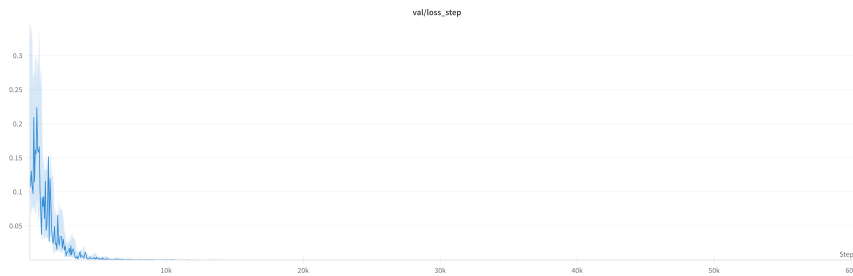


Figure 1: Loss plot for the derivative prediction task

This reflects also in a successful evaluation on the test set, where predictions are very close to the nominal model, with very few exceptions. Also the integration in the MPC is successful, achieving nearly identical results to the nominal model.

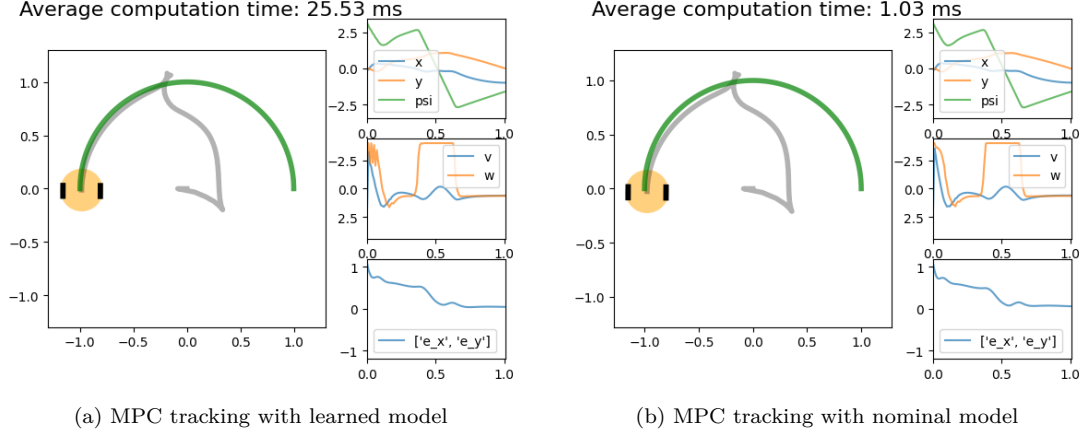


Figure 2: Derivative prediction evaluation

In these plots, the MPC simulation has a discretization timestep of 50 ms, in order to make the neural-mpc controller feasible, since it has higher computational demands than the nominal model. Of course, in this ideal setting, the nominal model performs better in computational terms, but the neural model is potentially more robust to disturbances and model errors, especially if we were to inject noise in it during the training process. The neural model is also potentially more scalable, since it can be trained on a variety of more complex models, while potentially keeping the same computational cost.

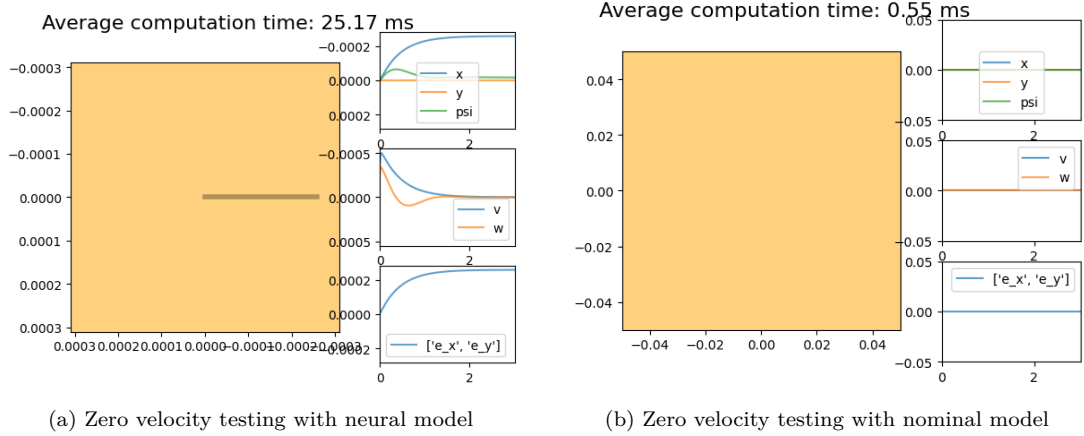


Figure 3: Derivative prediction evaluation

In figure 3 we can see that the neural model has some issues when the desired tracking velocity is zero. The neural mpc has a imperceptible drift, while the

nominal model is, of course, completely still. This derives from the intrinsic data-related nature of neural model, which can't captivate the differential equations profoundly.

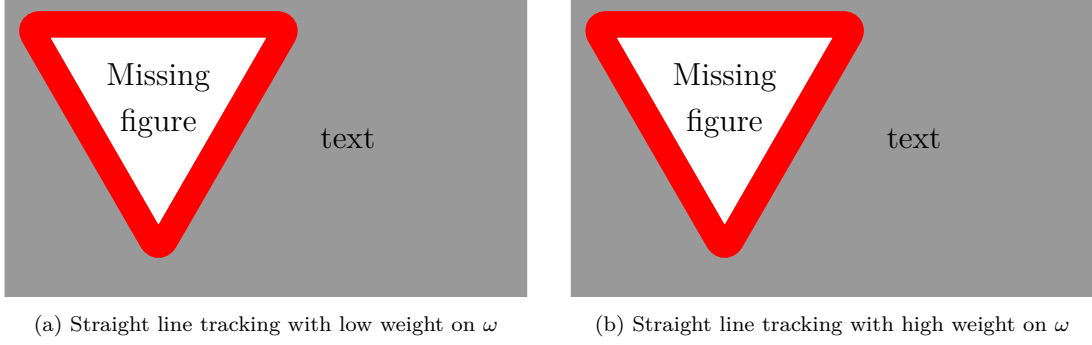


Figure 4: Testing two weight configurations when tracking a straight line

5.2 State Prediction

The loss for the state prediction task is depicted in figure 5. The trend is very similar to before.

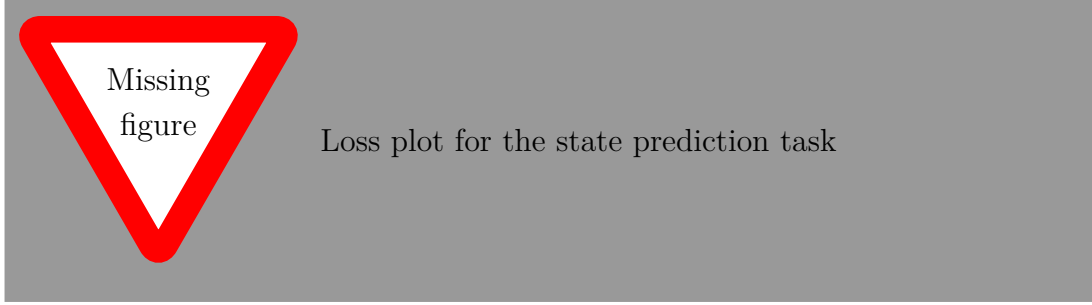


Figure 5: Loss plot for the state prediction task

Similarly to before, this reflects positively in the evaluation on the test, but there are some edge cases where dictions are not as accurate as before. This probably comes from the fact that we use Euler integration to propagate the nominal system dynamics.

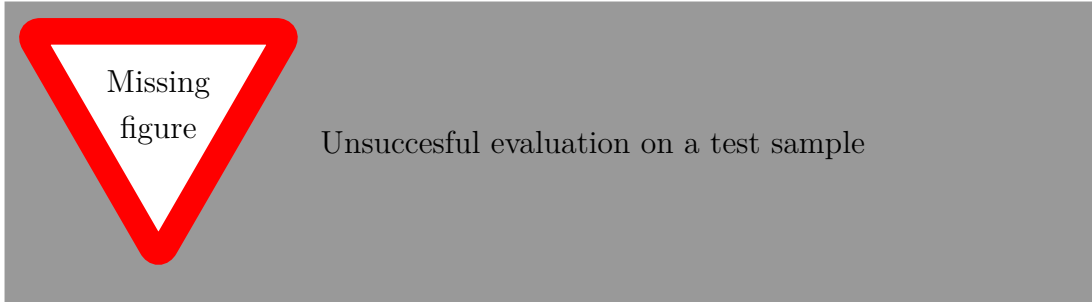


Figure 6: Unsuccessful evaluation on a test sample

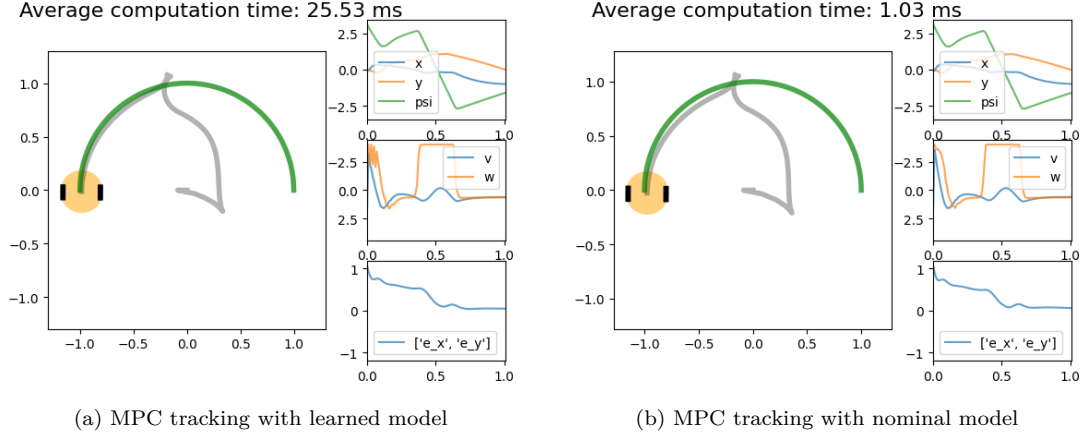


Figure 7: state prediction evaluation

The neural state prediction mpc performs better in computational terms, probably because acados handles better the discrete integration rather than ERK or Euler. On the other hand, the mpc weights need to be higher in order

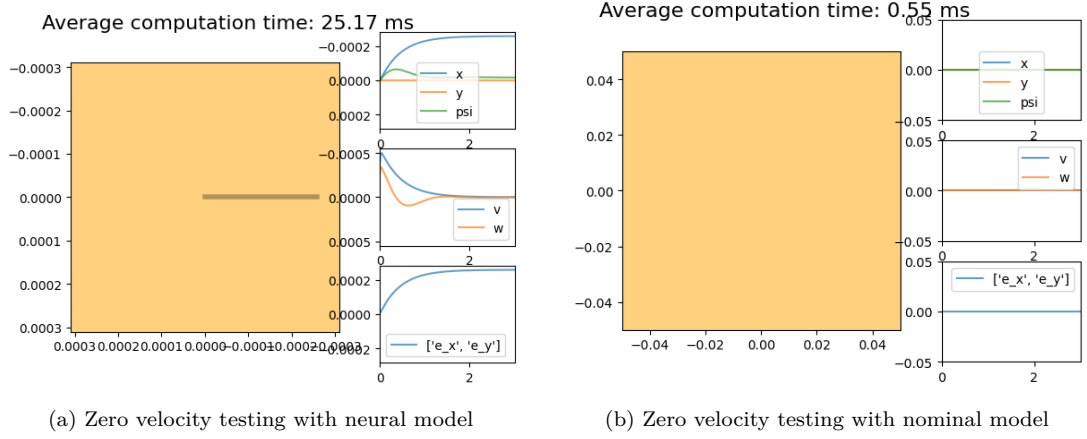


Figure 8: Derivative prediction evaluation

In figure 8 we can see that the neural model has some issues when the desired tracking velocity is zero. The neural mpc has a imperceptible drift, while the nominal model is, of course, completely still. This derives from the intrinsic data-related nature of neural model, which can't captivate the differential equations profoundly.

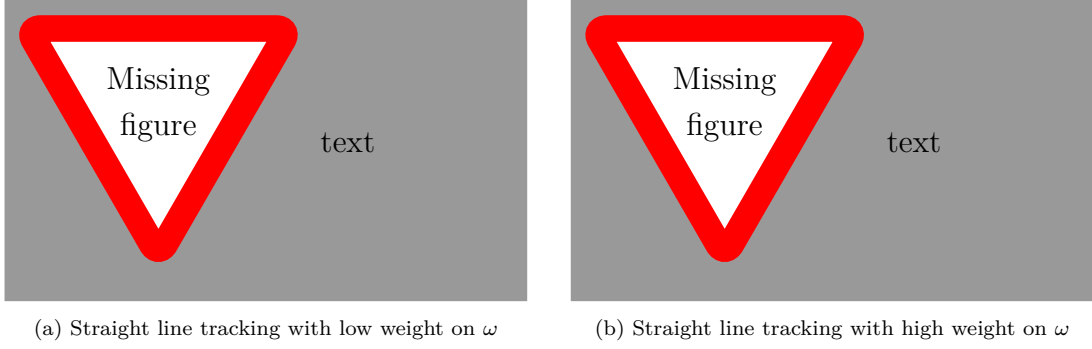


Figure 9: Testing two weight configurations when tracking a straight line

5.3 State Prediction with Physics Loss

The loss for the state prediction task is depicted in figure 5. The trend is very similar to before, with a quick

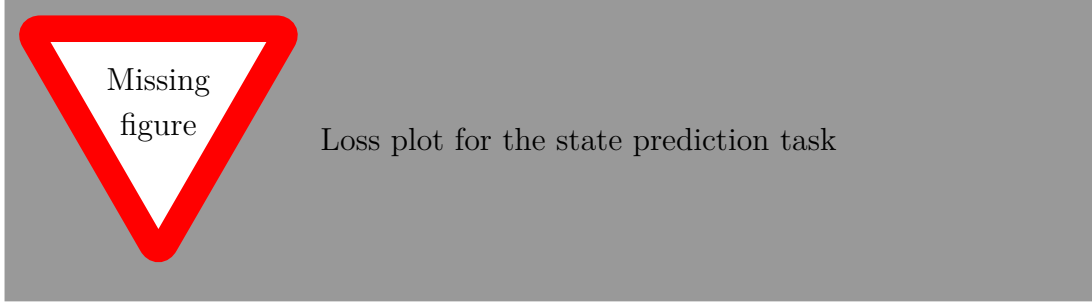


Figure 10: Loss plot for the state prediction task

This reflects also in a successful evaluation on the test set, where predictions are very close to the nominal model, with very few exceptions. Also the integration in the MPC is successful, achieving nearly identical results to the nominal model.

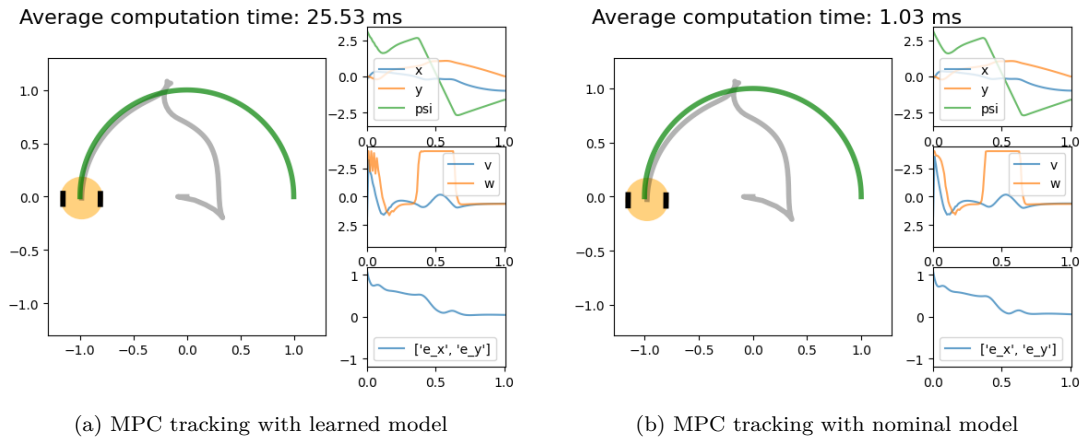


Figure 11: pinn prediction evaluation

In these plots, the MPC simulation has a discretization timestep of 50 ms, in order to make the neural-mpc controller feasible, since it has higher computational demands than the nominal model. Of course, in this ideal setting, the nominal model performs better in computational terms, but the neural model is potentially more robust to disturbances and model errors, especially if we were to inject noise in it during the training process. The neural model is also potentially more scalable, since it can be trained on a variety of more complex models, while potentially keeping the same computational cost.

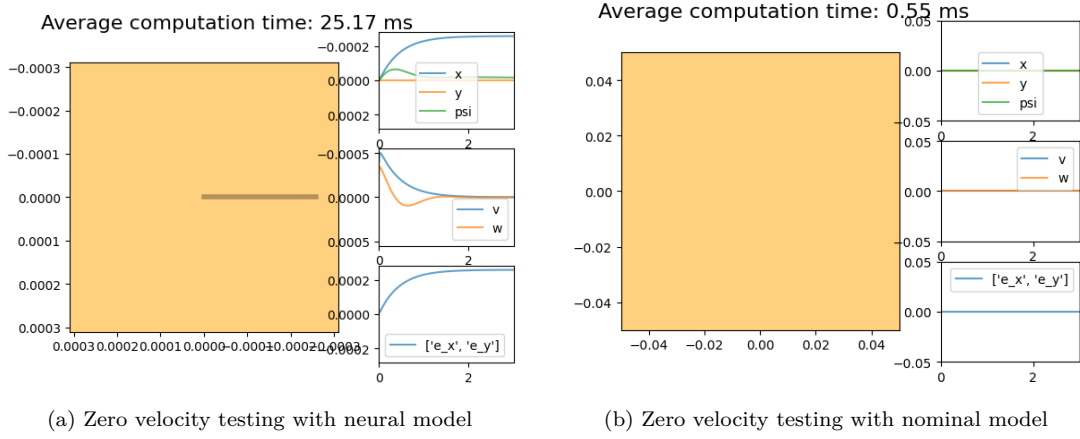


Figure 12: Derivative prediction evaluation

In figure 12 we can see that the neural model has some issues when the desired tracking velocity is zero. The neural mpc has a imperceptible drift, while the nominal model is, of course, completely still. This derives from the intrinsic data-related nature of neural model, which can't captivate the differential equations profoundly.

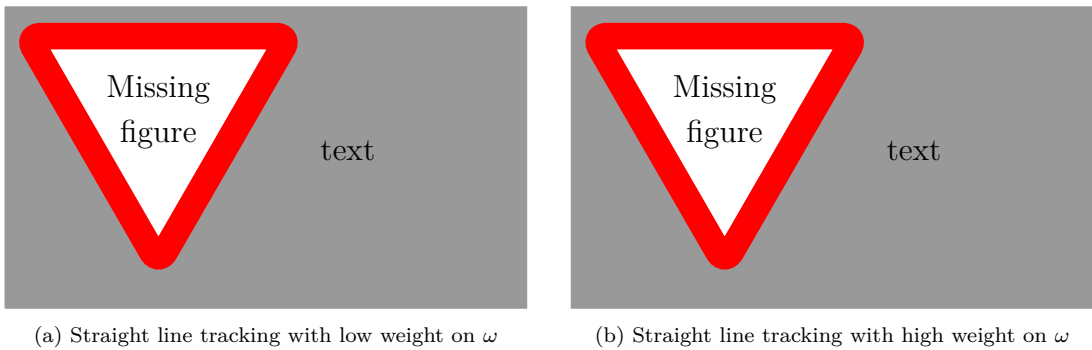


Figure 13: Testing two weight configurations when tracking a straight line

6 Conclusions

In this project we wanted to explore the possibility to integrate Pytorch-learned models within CasADI and Acados to provide a learning based model predic-

tive controller. This approach allows to exploit the vast adaptability of neural networks to many domains (including robotics) while exerting some control over the system through MPC, which allows to impose constraints. We have shown that this approach is feasible, while presenting also some limitations. By far, the main limitation is given by the intrinsic nature of neural networks, which don't take into account the actual physical meaning of the equations governing the dynamical system. This can be partially solved by using a physics-informed neural network, which we did for the unicycle. Another problem is given by how the neural network is integrated within the MPC. The optimization problem has its own computational graph, and the neural network has its own too, so these two graphs need to be merged. This problem was solved by using L4Casadi. Of course, this is just an exploratory project and many problems remain to be solved.

Add a
para-
graph
on the
future
work